Empacotamento de Pacotes em Frotas Heterogêneas: Modelagem VSBPP Bidimensional, Solução Exata e Método Alternativo sobre Dados Reais de Roteirização

Guilherme de Abreu Barreto, Eduarda Almeida Garrett de Carvalho, Gabriel Demba, Nicholas Eiti Dan, Wiltord Nyakeruma Mosingi

***Resumo*—Consolidar pacotes em menos viagens é uma das alavancas de custo mais diretas da distribuição física: em uma frota heterogênea, a decisão deixa de ser "quantos compartimentos" e passa a ser "quantos caminhões de cada tipo", cada um oferecendo dois recursos escassos — área de piso e peso — a um custo fixo por viagem proporcional à distância da rota. Formulamos esse problema como um empacotamento em caixas de tamanho e custo variáveis (VSBPP) bidimensional, com custo fixo por caminhão, e o instanciamos sobre o conjunto público *Large-Scale Route Optimization* (licença MIT), derivado do acelerador homônimo da Microsoft: 4.635 pacotes distribuídos em 61 rotas a partir de um único centro de distribuição e uma frota de quatro tipos de caminhão com capacidade de área, capacidade de peso e custo por quilômetro explícitos. Adotamos a política de viagem direta, que transforma cada rota em uma instância independente de VSBPP e faz do ótimo do empacotamento um limite superior para o custo do roteamento subjacente. O modelo é um programa linear inteiro misto com restrições de quebra de simetria; o plano experimental prevê solução exata por um solver comercial (Gurobi) e um de código aberto (HiGHS), comparando status, valor da função objetivo, tempo de processamento e gap de otimalidade, além de um método alternativo (heurística construtiva e meta-heurística) para as instâncias de maior porte. Os resultados experimentais estão em elaboração e serão incorporados à versão final.**

***Index Terms*—bin packing, empacotamento com caixas de tamanho variável, empacotamento bidimensional, frota heterogênea, programação inteira mista, consolidação de carga**

I. Introdução

O custo de uma cadeia de suprimentos é decidido, em boa parte, pela **composição da carga**: quantos veículos partem, de que tamanho, e com qual fração de sua capacidade ocupada. Consolidar pacotes que seguem a mesma origem e o mesmo destino em menos viagens é, por isso, uma das alavancas operacionais mais diretas de custo — e uma das mais difíceis, porque o número de planos de carregamento cresce de forma combinatória com o número de pacotes. A literatura de pesquisa operacional trata essa decisão como um problema de otimização combinatória: Dzalbs e Kalganova aplicaram colônia de formigas à consolidação de pedidos em reservas de frete de uma tabela tarifária real [1], e aceleradores industriais de roteirização transformam o mesmo núcleo combinatório em produto, resolvendo instâncias de milhares de pedidos [2], [3]. Sob a terminologia logística, o que esses trabalhos enfrentam é um **problema de empacotamento**.

Formalmente, o problema do empacotamento em caixas (*bin packing problem*, BPP) distribui itens de tamanho w_i em caixas de capacidade C, minimizando o número de caixas utilizadas. É NP-difícil [4], [5], de modo que não se pode esperar um algoritmo exato polinomial no pior caso. O que a teoria oferece são garantias de aproximação — a heurística *first fit decreasing* (FFD) nunca usa mais que $11/9$ do ótimo assintoticamente [6] — e limites inferiores apertados, que dão a régua para medir o quanto uma solução factível está longe do ótimo [7]. Em logística, essa régua é o que distingue uma política de carregamento defensável de uma simples regra de bolso.

Na prática, porém, as caixas não são iguais: frotas são heterogêneas e o preço de uma viagem depende do tipo de veículo. Na variante de caixas de tamanho e custo variáveis (VSBPP), as caixas vêm em classes com capacidades e custos distintos e o objetivo passa a ser o custo total das caixas usadas [8], [9]; nas versões de custo generalizado, cada classe tem seu próprio custo e ainda assim existem esquemas de aproximação assintóticos [10], [11]. Essa é a variante que interessa a um operador logístico: a decisão não é apenas "quantos caminhões", mas também "**quais** caminhões".

A isso soma-se uma dificuldade que o contexto físico impõe e que a literatura unidimensional não vê. Um caminhão oferece dois recursos escassos e independentes — área de piso e capacidade de peso —, e um carregamento só é viável se respeitar ambos; o problema é, portanto, um empacotamento vetorial bidimensional [12], [13]. Empacotamento vetorial é estritamente mais difícil que o unidimensional: para duas ou mais dimensões não existe esquema de aproximação polinomial assintótico, a menos que P = NP [14], enquanto em uma dimensão ele existe tanto para o BPP clássico [15], [16] quanto para o VSBPP [10], [11]. Não há, então, algoritmo com garantia de qualidade para o nosso caso: restam a programação inteira mista nas instâncias pequenas e heurísticas desenhadas para o domínio nas grandes.

A programação inteira mista, por seu turno, cobra um preço próprio. Formulações de empacotamento são altamente simétricas — compartimentos são intercambiáveis —, e a simetria degrada tanto a relaxação linear quanto o *branch-and-bound* [17], [18]. A resposta consolidada na literatura é

Instituto de Ciências Matemáticas e de Computação (ICMC), Universidade de São Paulo. Autor correspondente: de_abreu@usp.br. E-mails: eduda.agc@usp.br (E. A. G. de Carvalho); gabrielgaby@usp.br (G. Demba); nicholas.dan@usp.br (N. E. Dan); wiltordmosingi@usp.br (W. N. Mosingi).

quebrar a simetria com restrições e técnicas dedicadas [19], [20], cujo efeito prático já foi medido no BPP unidimensional [21]. Boa parte do trabalho de modelagem relatado adiante está aí.

A instância que estudamos vem do conjunto público *Large-Scale Route Optimization*, espelho em licença MIT do acelerador homônimo da Microsoft [2], [3]. Ele descreve um cliente de manufatura que distribui pedidos a partir de um único centro de distribuição para 61 destinos e fornece a frota em quatro tipos de caminhão, cada um com área de piso, capacidade de peso e custo por quilômetro explícitos. São 4.635 pacotes em 365 pedidos e 61 pares origem–destino, com pacotes de 0,64 a 9,72 m² e de 46 a 3.130 kg — nenhum excede o menor caminhão. Trata-se de uma **amostra publicada**, declarada pelos autores como "inspirada em um cenário real" de um cliente de manufatura, e não de dados brutos da empresa; isso basta para o exercício, mas precisa ser dito.

O acelerador resolve o problema de roteirização de veículos (VRP), no qual um caminhão atende várias paradas. Aqui adotamos a política oposta: **viagem direta**, uma parada por viagem. Com isso o roteamento degenera no carregamento de uma rota, cada par origem–destino vira uma instância independente de VSBPP, e o ótimo desse empacotamento passa a ser um **limite superior** para o custo do VRP — não o seu ótimo. A escolha é deliberada: isola a decisão de composição de frota, que é onde a modelagem tem algo a dizer, e deixa o ganho do compartilhamento de veículos como uma quantidade explícita a ser medida depois.

Este trabalho persegue, então, quatro objetivos: (i) formular a consolidação de carga como um VSBPP bidimensional de custo fixo, declarando e justificando conjuntos, parâmetros, variáveis e restrições; (ii) resolvê-lo exatamente em instâncias pequenas com dois solvers — um comercial e um de código aberto —, reportando status, valor da função objetivo, tempo de processamento e gap de otimalidade, e discutindo a diferença de desempenho entre eles; (iii) implementar um método de solução alternativo (heurística construtiva e, para as instâncias maiores, meta-heurística); e (iv) comparar criticamente as duas abordagens quanto à qualidade da solução, ao esforço computacional e à aplicabilidade prática. A seção de Metodologia apresenta a base de dados e a formulação; as seções de solução via solvers, de método alternativo e de conclusão serão incorporadas à medida que os experimentos forem concluídos.

II. Metodologia

Esta seção descreve de onde vêm os números e como o problema é formulado. A primeira parte apresenta a base de dados, a frota e as instâncias que dela derivamos; a segunda descreve o pré-processamento que transforma as tabelas brutas nas instâncias resolvidas; a terceira declara o modelo de otimização inteira mista que será resolvido.

A. Base de dados

1) Fonte e esquema: O conjunto é distribuído no Kaggle sob licença MIT [3] e é espelho dos dados de amostra do acelerador da Microsoft [2]; os arquivos usados estão versionados em `data/raw/`:

- `order_large.csv` — **4.635 pacotes** em 365 pedidos, distribuídos em **61 pares origem–destino**; é a instância principal;
- `order_small.csv` — 10 pacotes em 5 pedidos e 2 pares origem–destino, usado como caso de teste mínimo;
- `distance.csv` — matriz de distâncias entre cidades, **em metros** (apesar do nome da coluna, `Distance(M)`; é o que converte custo por km em custo por viagem);
- `trucks.csv` — a frota, transcrita de `src/core/structure.py` do repositório de origem; o README do projeto lista uma tabela mais antiga e inconsistente, e o código é a fonte autoritativa.

As colunas `Area` e `Weight` vêm escaladas por 10^4; todos os valores citados neste relatório já estão convertidos para m² e kg. Das 20 colunas disponíveis, `Available_Time` e `Deadline` alimentam as restrições temporais do modelo e `Danger_Type` não é utilizada.

B. Análise estatística exploratória

1) Frota: A frota tem quatro tipos de caminhão, cada um definido pela área de piso A_t, pela capacidade de peso W_t e pelo custo C_t por quilômetro — os três parâmetros que, multiplicados pela distância da rota, dão o custo fixo de cada viagem. A tabela a seguir reúne esses valores e dois indicadores derivados que serão úteis na discussão do modelo.

O custo por unidade de capacidade é monotônico no porte do veículo: cai de US$ 0,332/m² no caminhão de 7,6 m para US$ 0,248/m² no de 16,5 m (e de US$ 0,00069/kg para US$ 0,00037/kg), de modo que "maior é mais barato por unidade". O indicador oposto, porém, não é monotônico: a densidade máxima do caminhão **cai** com o porte, de 793 kg/m² no de 12,5 m para 671 kg/m² no de 16,5 m e 483 kg/m² no de 7,6 m. As duas tendências apontam para direções diferentes, e é essa tensão que faz da escolha de classe uma decisão de otimização, não uma regra de bolso.

Nenhum pacote excede a maior classe — o item mais pesado tem 3.130 kg e o maior em área, 9,72 m², contra 27.000 kg e 40,25 m² —, de modo que a frota, por si só, **não torna nenhuma instância inviável**: ela encarece a solução, não a impede.

A densidade é o parâmetro que decide qual dos dois recursos se esgota primeiro: uma carga cuja densidade fica abaixo de W_t/A_t enche a área do caminhão antes de atingir o peso máximo, e acima desse limiar acontece o contrário. A Fig. 1 mostra a distribuição item a item. Tomada item a item, a mediana é de **589 kg/m²** — à esquerda das razões W_t/A_t dos caminhões de 16,5 m (671), 12,5 m (793) e 9,6 m (669), e à direita da do caminhão de 7,6 m (483) —, embora a cauda se estenda bem acima delas. Somando a carga de uma rota, a densidade agregada cai para **554 kg/m²**, e é esse valor que decide o regime de quase todas as rotas.

2) Pacotes: Os 4.635 pacotes não são 4.635 objetos distintos do ponto de vista do modelo. O `Item_ID` é único por construção, mas as combinações de **(área, peso)** são apenas **134**; os dez tipos mais frequentes concentram **37%** dos itens e o mais frequente aparece 233 vezes, enquanto 12 tipos ocorrem uma única vez. Se a identificação incluir também as janelas

Comprimento	A_t (m^2)	W_t (kg)	C_t (US$/km)	C_t/A_t (US$/m^2)	C_t/W_t (US$/kg)	W_t/A_t (kg/m^2)
16,5 m	40,25	27.000	10,00	0,248	0,00037	671
12,5 m	30,25	24.000	9,00	0,298	0,00038	793
9,6 m	20,93	14.000	6,50	0,311	0,00046	669
7,6 m	16,56	8.000	5,50	0,332	0,00069	483

Tabela I: Frota do conjunto *Large-Scale Route Optimization*: capacidade de área e de peso, custo por quilômetro e indicadores derivados. Custos calculados a partir de `cost_per_km` e capacidades de `area_capacity_m2` e `weight_capacity_kg` de `trucks.csv`; o comprimento identifica a classe.

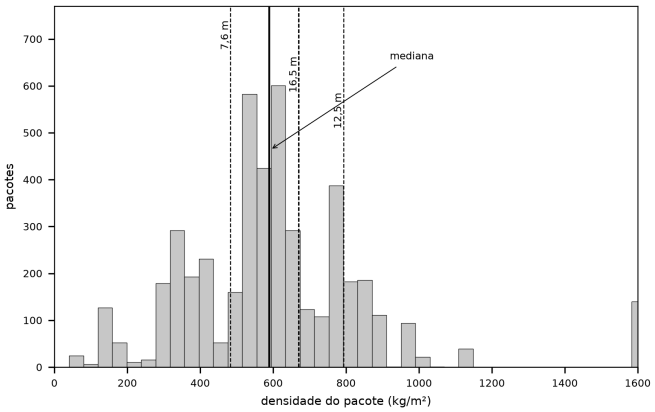


Fig. 1: Densidade dos pacotes de `order_large.csv`, uma observação por item (kg/m²). Linhas tracejadas: razão W_t/A_t de cada tipo de caminhão — abaixo dela a área se esgota antes do peso. Linha cheia: mediana dos itens (589 kg/m²). A do caminhão de 9,6 m (669 kg/m²) praticamente coincide com a do de 16,5 m.

de tempo, o número de **classes** (área, peso, disponibilidade, prazo) sobe para **235** — e, por rota, a variedade é ainda menor: nenhuma rota passa de 20 classes. É essa duplicação que permite descrever a instância por **grupos de pacotes idênticos**, com uma variável de contagem por grupo em vez de um binário por item — a decisão de modelagem adotada na formulação.

Em termos de tamanho, os pacotes ocupam de 0,64 a 9,72 m² (mediana 1,25) e pesam de 46 a 3.130 kg (mediana 1.000) — ambos com cauda longa à direita, como mostra a Fig. 2. As duas distribuições são assimétricas na mesma direção, o que já sugere que área e peso não variam de forma independente.

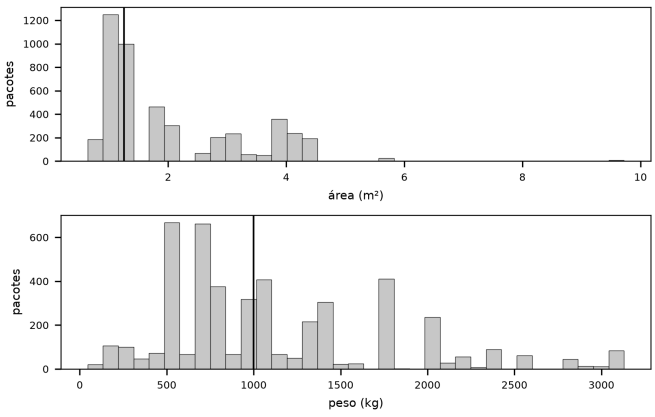


Fig. 2: Distribuição dos 4.635 pacotes de `order_large.csv`: área (m²) e peso (kg), em painéis empilhados. Linha vertical: mediana (1,25 m² e 1.000 kg). As duas variáveis têm cauda longa à direita.

Área e peso crescem juntos, mas **longe de proporcionalmente**: a correlação é de 0,68 (Pearson) e 0,72 (Spearman), e a razão entre eles — a densidade — varia de 40 a 1.623 kg/m², com mediana 589. A Fig. 3 mostra esse leque: sobre uma mesma área, o peso observado pode variar por um fator próximo de 10.

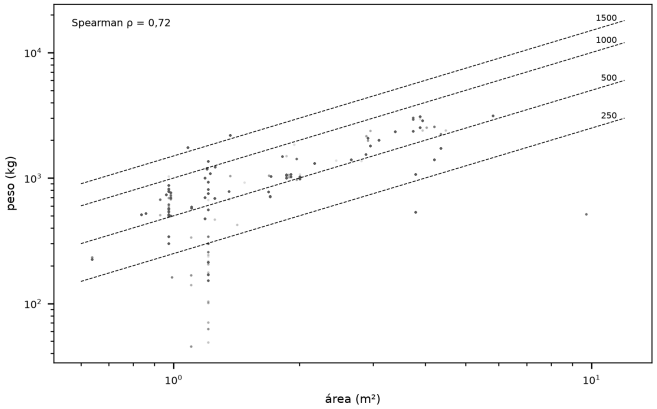


Fig. 3: Relação entre área e peso dos pacotes (escalas logarítmicas, um ponto por item). As retas tracejadas marcam densidades constantes; a faixa ocupada mostra que a densidade varia cerca de 40 vezes entre os itens. Correlação de Spearman: 0,72.

Essa dispersão tem consequência direta no modelo: como a densidade não é constante, **não existe um fator fixo** que converta área em peso, e as duas restrições de capacidade (R2 e R3) mordem em proporções diferentes conforme a rota — é o que justifica tratar a instância como bidimensional em vez de escolher uma das duas dimensões como proxy.

3) Pedidos: Os 4.635 pacotes chegam em **365 pedidos** (`Order_ID`), de tamanhos muito desiguais: de 1 a 39 pacotes, com mediana de 10 (p10 de 2, p90 de 29) e peso mediano de 16,2 t. Um pedido é, ainda, **homogêneo**: em 86,6% deles há um único tipo (área, peso), em 11,8% dois, em 1,4% três e em 0,3% quatro. A Fig. 4 resume as duas coisas — o volume por pedido e a diversidade de tipos dentro dele.

O tamanho desigual dos pedidos produz uma concentração moderada: o índice de Gini dos pacotes por pedido é de **0,42** — os 10% maiores pedidos carregam 28% dos pacotes, e a metade menor responde por 22% —, como mostra a Fig. 5. Nada parecido com uma distribuição de cauda pesada: há pedidos grandes, mas não há alguns poucos pedidos dominando o conjunto.

No tempo, a oferta é concentrada e a demanda é espalhada. Todos os 4.635 pacotes ficam disponíveis em **três dias** (5 a 7 de abril de 2022), em lotes semelhantes — 1.495, 1.399 e 1.741 pacotes, cerca de 1,7 mil t por dia —, enquanto os prazos se distribuem por **oito dias** (6 a 13 de abril), com pico em 9 de abril (106 pedidos). A janela entre disponibilidade e prazo tem mediana de 2,5 dias e máximo de 6. A Fig. 6 compara as duas distribuições.

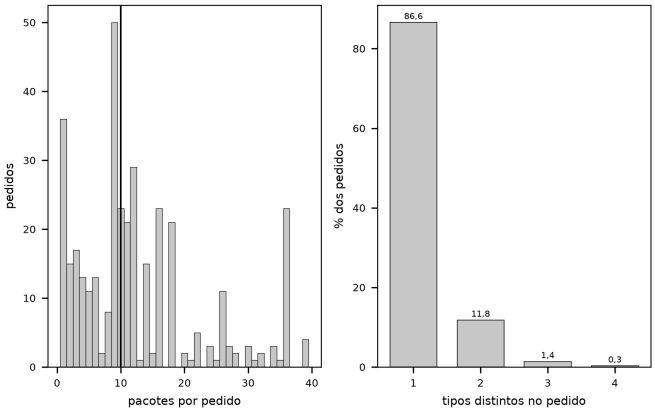


Fig. 4: Volume e diversidade dos 365 pedidos de `order_large.csv`. Esquerda: número de pacotes por pedido (mediana 10; linha vertical). Direita: percentual de pedidos segundo o número de tipos (área, peso) distintos que contêm — 86,6% são homogêneos.

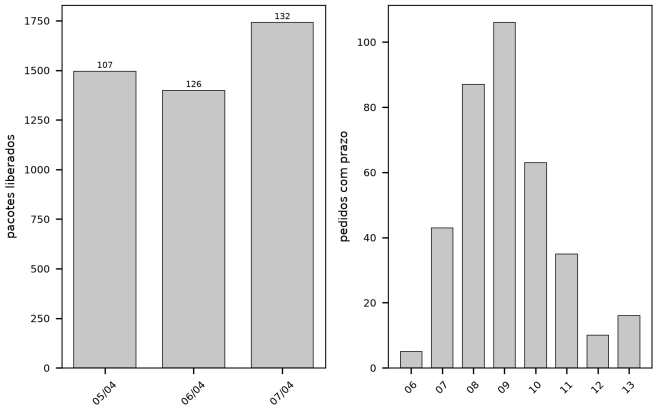


Fig. 6: Distribuição temporal dos pedidos de `order_large.csv`. Esquerda: pacotes liberados por dia de `Available_Time` (o número sobre cada barra é o de pedidos daquele dia). Direita: pedidos por dia de `Deadline`. A liberação ocupa três dias e os prazos, oito.

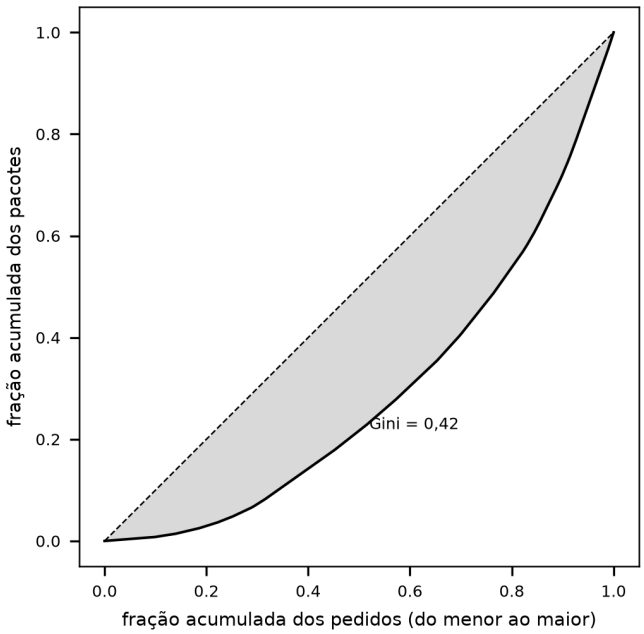


Fig. 5: Curva de Lorenz dos tamanhos de pedido (pacotes por `Order_ID`). A diagonal representa igualdade perfeita; a área entre a curva e a diagonal é o índice de Gini (0,42). Os 10% maiores pedidos concentram 28% dos pacotes.

descontando $t_{\text{viagem}} = d/v$, a folga mediana cai para **30 h** e **73 dos 4.635 itens** ficam com menos de 4 h. Nesses casos o instante de partida é praticamente forçado, e a única liberdade que resta ao modelo é o emparelhamento — é aí que R6–R8 mordem. A Fig. 7 mostra as duas distribuições.

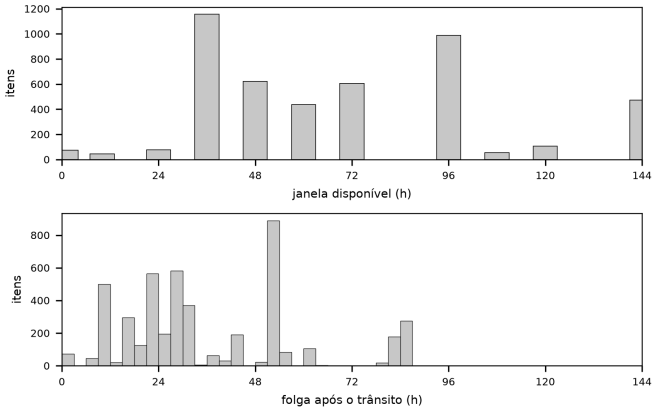


Fig. 7: Tempo disponível por item, em painéis empilhados e na mesma escala (0 a 144 h). Acima: janela total entre `Available_Time` e `Deadline` (1 a 144 h, mediana 60 h). Abaixo: folga que sobra depois de descontar o tempo de viagem na rota (mediana 30 h; 73 itens com menos de 4 h) — a distribuição desloca-se para a esquerda e deixa de usar o topo da escala.

A concentração também é espacial — mas **heterogênea**: dos 61 destinos, 28 recebem carga em um único dia, 16 em dois e 17 nos três, e a mediana do dia dominante é de 89,5% da carga do destino. O padrão, porém, se inverte com o volume: entre os dez maiores, nove usam os três dias (a carga de `City_58` reparte-se em 31%, 42% e 27%), enquanto os destinos pequenos tendem a concentrar tudo num dia só.

Esse resultado tem consequência direta para o modelo: como os dias consecutivos de liberação distam **24 h**, muito acima de $\Delta = 4$ h, um caminhão não consegue misturar pacotes liberados em dias diferentes. O recorte rota × dia não fragmenta a instância artificialmente — ele reflete a própria janela de consolidação.

A janela entre disponibilidade e prazo mede de 1 a 144 h (mediana de 60 h), mas o trânsito consome parte dela:

Dois fatos fecham a leitura dos pedidos como unidade de análise. Primeiro, `Available_Time` e `Deadline` são **constantes dentro de cada pedido** nos 365 casos: o pedido é a menor unidade temporal do conjunto, e não o pacote. Segundo, como um pedido é quase sempre homogêneo, a diversidade de tipos que aparece numa rota vem da **mistura de pedidos**, não da composição interna de cada um. É por isso que o recorte experimental natural é rota × dia de disponibilidade — o que devolve 111 instâncias não vazias, tratadas na próxima subseção.

4) Rotas: Os dados cobrem 62 cidades, das quais uma é o centro de distribuição (`City_61`) e 61 são destinos; todas as viagens consideradas aqui são diretas, do centro a um destino. A topologia da rede aparece na Fig. 8. `distance.csv` traz a matriz completa e simétrica — 62 cidades, 1.891 pares —,

desenhada em cinza; sobre ela, em preto, estão as 61 rotas diretas que partem do centro de distribuição, com espessura proporcional ao número de pacotes (de 1 a 468). As posições **não são geográficas**: são a projeção em duas dimensões da própria matriz de distâncias, obtida por escalonamento multidimensional clássico, de modo que a distância no desenho aproxima a distância entre as cidades. O que a figura mostra é a estrutura de *hub*: toda a carga entra e sai por um único nó, e `City_58`, o destino mais carregado, é também um dos mais distantes.

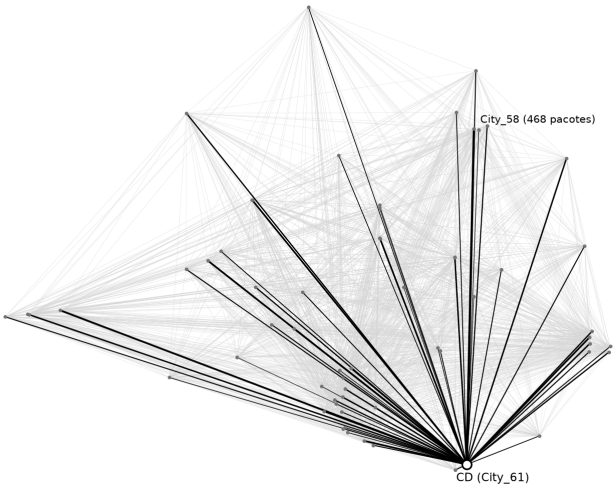


Fig. 8: Rede de rotas do conjunto *Large-Scale Route Optimization*. Linhas cinza: os 1.891 pares de `distance.csv` (62 cidades). Linhas pretas: as 61 rotas diretas a partir do centro de distribuição (`City_61`), com espessura proporcional ao número de pacotes da rota (1 a 468). Posições por escalonamento multidimensional da matriz de distâncias — é uma projeção, não um mapa geográfico.

A consequência agregada do caráter bidimensional aparece na Fig. 9, que compara, rota a rota, os caminhões que a área exigiria com os que o peso exigiria se a frota fosse só de veículos de 16,5 m. Com esse veículo, o limite por área é o mais apertado em **18 das 61 rotas**, empata em 38 e é mais folgado em apenas 5; no total, a frota exigiria **270 caminhões por área contra 226 por peso**. É por isso que a formulação não pode ser unidimensional.

O caso `City_61` → `City_58` (1.754 km, 468 pacotes, 1.163 m² e 418 t) é o extremo da assimetria: seriam 16 caminhões se apenas o peso contasse e 29 se apenas a área contasse.

Recortando as rotas por dia de disponibilidade, obtêm-se 111 instâncias não vazias, com mediana de 31 pacotes (p90 de 86 e máximo de 198) — o que define os três portes de instância do plano experimental: pequenas (resolução exata sem limite de tempo), médias (exata com limite) e grandes (apenas heurística).

C. Pré-processamento

O conjunto bruto é uma tabela de pedidos; o modelo precisa de instâncias — e de instâncias pequenas. Esta seção descreve as etapas que transformam um no outro sem alterar o ótimo: normalização de unidades, recorte das instâncias e redução do

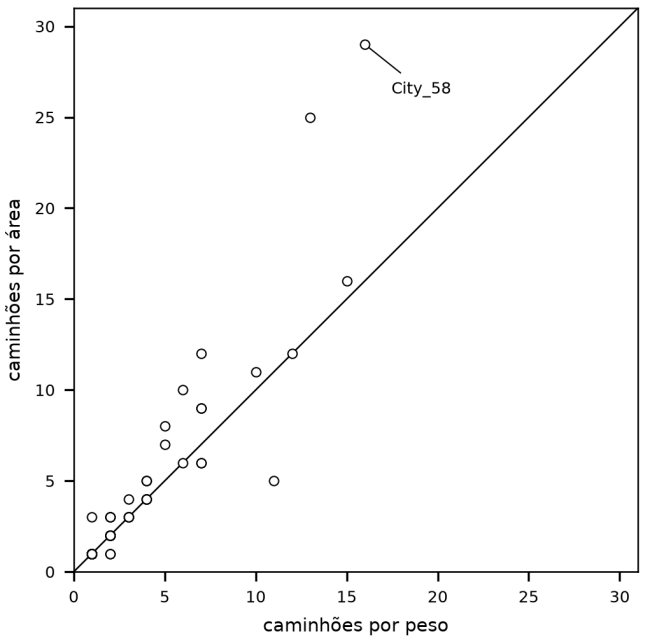


Fig. 9: Caminhões por rota, se a frota fosse só de veículos de 16,5 m: limite inferior por área ($\lceil \sum_i a_i/A_t \rceil$) contra limite mínimo por peso ($\lceil \sum_i w_i/W_t \rceil$). Acima da diagonal, a área exige mais caminhões que o peso; abaixo, o contrário; sobre ela, os dois limites coincidem.

modelo. O agrupamento de pacotes idênticos, que também reduz a instância, é parte da formulação e está descrito na seção de modelagem.

O que segue é executado pelos blocos de código abaixo. Eles ficam ocultos no PDF (`echo: false`), mas permanecem no `.qmd`: cada bloco é precedido do que faz e seguido do que produziu. O resultado é gravado em `data/pre-processed/` e reaproveitado nas execuções seguintes.

1) Normalização e recorte: As colunas `Area` e `Weight` vêm escaladas por 10^4 e precisam virar m² e kg; as distâncias, de metros para km. Os carimbos `Available_Time` e `Deadline` são lidos como datas e convertidos em **horas contadas a partir da menor disponibilidade da instância**, o que mantém os números pequenos e a constante M da modelagem no menor valor possível — $M = \max_g dl_g - \min_g av_g + t_{\text{viagem}} + \Delta$.

O primeiro bloco carrega as três tabelas já usadas na análise exploratória, converte as unidades e prepara a chave de instância (destino, data de disponibilidade). Ele também decide se o resultado anterior pode ser reaproveitado: o cache vale enquanto `data/pre-processed/` for mais novo que `order_large.csv`, `distance.csv` e `trucks.csv`.

O segundo bloco percorre as combinações (destino, dia de disponibilidade) e, para cada uma, faz três coisas: converte os tempos para horas relativas à origem da instância; agrupa os pacotes idênticos — mesma área, peso, disponibilidade e prazo — em grupos com multiplicidade n_g; e guarda as arrays de cada instância para o bloco seguinte. O agrupamento é o que permite ao modelo trabalhar com contagens em vez de um binário por pacote.

O recorte é **rota × dia de disponibilidade**, e é exato, não uma conveniência: como os dias consecutivos de liberação

distam 24 h e um caminhão só consolida pacotes cuja disponibilidade difere em até $\Delta = 4$ h, nenhuma solução mistura dias.

O recorte devolve 111 instâncias não vazias — mediana de 31 pacotes, p90 de 86 e máximo de 198 —, que é a base dos três portes do plano experimental. Nenhum grupo tem $av_g + t_{\text{viagem}} > dl_g$ (0 casos): a triagem de inviabilidade não corta instância alguma. O agrupamento comprime as 4.635 linhas de pacotes em 239 grupos (instância × classe).

2) Redução do modelo: O terceiro bloco aplica os cortes que preservam o ótimo:

- **Filtro de tipos.** Um tipo só entra na instância se comporta ao menos um pacote ($A_t \geq \max_g a_g$ e $W_t \geq \max_g w_g$) e não é dominado por outro tipo disponível na mesma instância — com custo por viagem $C_t = c_t d$.
- **Teto de slots.** m não precisa ser n: como todo caminhão custa ao menos $c_{\min} d$, uma solução factível de custo UB limita $m \leq \lfloor \text{UB}/(c_{\min} d) \rfloor$. O UB vem de uma construção gulosa (primeira adaptação em ordem decrescente de área, respeitando área, peso e janela). Cada caminhão dessa construção paga o tipo mais barato que o comporta, de modo que $\text{UB} = \sum_b \min\{c_t : A_t \geq a_b,\ W_t \geq w_b\}\, d$, somado sobre os caminhões b; o número de caminhões usados fica registrado como diagnóstico, nunca como teto — usá-lo seria **inválido**, porque o ótimo pode usar mais caminhões menores e sair mais barato, enquanto o teto de custo é seguro, embora frouxo.
- **Fechamento por inspeção.** Quando a carga inteira cabe num caminhão e a janela permite, o ótimo é o custo desse caminhão e não há MILP a resolver.

O bloco grava `instances.csv` (uma linha por instância, com os parâmetros do modelo), `groups.csv` (uma linha por grupo, com n_g) e um `meta.json` com o tempo de construção.

Os números que o bloco produziu: 44 das 111 instâncias fecham por inspeção, e o teto de custo é válido mas frouxo — na maior instância (City_58_2022-04-06, 198 pacotes), a construção heurística usa 10 caminhões e o teto permite 16 slots, num total de 465 slots nas 111 instâncias. Nenhuma instância passa de 11 grupos.

Corte	Efeito medido
Teto de slots pelo custo	maior instância (`City_58`, 198 pacotes): 10 caminhões na construção heurística, 16 slots; 465 slots para 111 instâncias
Filtro de tipos	nenhum par domina na frota global; vale por instância
Fechamento por inspeção	44 das 111 instâncias resolvidas sem solver

Tabela II: Cortes de pré-processamento e efeito medido. O primeiro é contado em slots; os dois últimos, em instâncias afetadas. Números produzidos pelos blocos de código desta seção.

Não há etapa de **rotulagem** de pacotes. Na versão item a item do modelo, a ordem dos índices importava — restrições de quebra de simetria davam ao pacote i apenas os slots $1..i$ —, e testamos cinco políticas de ordenação. Com o agrupamento não existe índice de pacote a escolher: o grupo carrega apenas a multiplicidade n_g, e a pergunta desaparece.

3) Custo do pré-processamento: O passo completo — leitura, normalização, recorte, agrupamento, filtro de tipos, teto de slots e fechamento por inspeção — levou **0,19 s** para construir as 111 instâncias. Como o resultado fica em `data/pre-processed/`, as execuções seguintes apenas o leem: nesta renderização o custo foi de 0,03 s (origem: cache). O tempo é desprezível diante do de otimização, mas o cache evita que qualquer ajuste no texto do relatório pague novamente a construção.

D. Modelagem Matemática

Em nosso trabalho, adaptamos o problema proposto pelo artigo de referência — a otimização do roteamento de veículos (*Vehicle Routing Problem*, VRP) — impondo um limite ao número de paradas por viagem. No problema original, admite-se que um veículo realize várias entregas ao longo do trajeto até o destino, o que permite reduzir ao mesmo tempo o número de veículos e a distância total percorrida [2], [3]. O carregamento dos pedidos é considerado, mas o custo por distância percorrida é o fator predominante na precificação das viagens. Ao limitar o número de paradas a **uma**, todas as viagens tornam-se diretas e o problema de roteamento degenera em um **problema de empacotamento com caixas de tamanho variável** (*Variable Sized Bin Packing Problem*, VSBPP) [8], [10], [11]. Noutras palavras, ao resolvermos o VSBPP, estipulamos o teto dos custos que o VRP pode reduzir.

O VSBPP difere do problema clássico de empacotamento em caixas por admitir compartimentos de capacidade variável. No caso do nosso dataset, os compartimentos são caminhões com distintas restrições de área e carga. No vocabulário do domínio: itens são **pacotes**; caixas são **caminhões** de quatro tipos (identificados por sua extensão); e a **instância** é uma rota $(\text{CD} \to d)$ em viagem direta para 61 destinos distintos.

Como cada pacote consome dois recursos do caminhão — área e carga —, a instância é **multidimensional** [12], [13]. Isso a torna mais difícil que o caso unidimensional. Um **esquema de aproximação polinomial assintótico** (APTAS) é uma família de algoritmos que, para todo $\varepsilon > 0$, garante solução a no máximo $(1+\varepsilon)$ vezes o ótimo mais uma constante, em tempo polinomial no número de itens. Para o empacotamento vetorial em duas ou mais dimensões tal esquema **não existe**, a menos que P = NP [14]; em uma dimensão ele existe tanto para o BPP clássico [15], [16] quanto para o VSBPP [10], [11].

Ser NP-difícil, porém, não torna o MILP inútil: BPP e VSBPP já são NP-difíceis [4] e solvers exatos resolvem instâncias grandes quando a formulação é boa. O que pesa aqui é a combinação de duas dimensões com a escolha da classe: não há garantia de aproximação e o esforço exato cresce rápido com o número de itens.

Esta seção formaliza um **VSBPP multidimensional** com frota heterogênea, instanciado no dataset de roteirização da Microsoft [2], [3]. A formulação segue os termos padrão de *bin packing* [8] [22] e acrescenta duas generalizações: as caixas vêm em **classes** com capacidade em **duas dimensões** e custo fixo por viagem; e cada item tem uma **janela de tempo** (disponibilidade e prazo).

1) Conjuntos:

- $G = \{1, \ldots, G\}$: **grupos de pacotes idênticos** — pacotes com a mesma área, peso, disponibilidade e prazo —,

cada grupo com multiplicidade n_g; o total de pacotes da instância é $n = \sum_g n_g$;
- $T = \{1, \ldots, 4\}$: tipos de caminhão disponíveis;
- $J = \{1, \ldots, m\}$: *slots* (posições) de caminhão disponíveis, com $m \leq n$.

A instância é definida **sobre grupos**, e não sobre pacotes. Isso não perde nada: pacotes de um mesmo grupo são indistinguíveis para todas as restrições — mesma área, mesmo peso e mesma janela —, de modo que a única informação que os distingue é **quantos** vão em cada caminhão. A decisão z_{gj} é, portanto, uma contagem, e a reconstrução de uma solução pacote a pacote é uma etapa de pós-processamento (distribuir as identidades dentro do grupo), sem efeito sobre custo ou viabilidade.

Um **slot** não é um caminhão específico, é uma **posição abstrata** que o modelo pode deixar vazia ou preencher com um caminhão de algum tipo. Como o **número** de veículos é justamente o que se quer decidir, criam-se m posições candidatas e deixa-se que y_{jt} indique se a posição j é um caminhão do tipo t; se nenhum tipo for atribuído, o slot permanece fechado (R4–R5). Os slots são intercambiáveis entre si — trocar dois produz a mesma solução física —, o que gera **simetria de permutação**: cada plano de carregamento tem até $m!$ representações de custo idêntico e de mesmo limitante [17]. A simetria não altera o valor ótimo, mas encarece a prova de otimalidade: cópias simétricas têm o mesmo limitante, então o *branch-and-bound* não consegue podar uma a partir do resultado da outra, e o esforço passa a acompanhar o número de rótulos em vez do número de soluções distintas [19]. A relaxação linear também fica degenerada, com vértices fracionários equivalentes [18], e as desigualdades válidas obtidas para uma cópia não se transpõem automaticamente às demais [17] — o que motivou as técnicas de quebra baseadas em órbitas [19], [20]. Por isso o modelo inclui a restrição de quebra **R10**, que fixa o padrão de abertura.

O agrupamento já elimina, por construção, a simetria **entre pacotes idênticos**: não há um índice por pacote para permutar. Resta a simetria entre slots, tratada na subseção de quebra de simetria.

2) Parâmetros: O *dataset* de referência nos fornece 20 colunas distintas, das quais **11** são incorporadas enquanto parâmetros em nossa modelagem. Esta correlação está descrita nas quatro tabelas 3 à 6, sobre as quais devemos fazer algumas considerações importantes.

a) Multiplicidade de pacotes idênticos:

A multiplicidade n_g não vem de coluna alguma: é a contagem de pacotes que compartilham a mesma quádrupla (área, peso, disponibilidade, prazo). No conjunto completo há 4.635 pacotes para 235 classes distintas — 239 grupos quando contados por instância —, e nenhuma instância passa de 11 grupos, o que encurta a formulação em uma ordem de grandeza (ver a análise de complexidade). Se todos os pacotes fossem distintos, teríamos $n_g = 1$ para todo g e a formulação abaixo coincidiria com a versão item a item.

b) Área enquanto uma grandeza vetorial:

Para cada pacote, o dataset fornece `Area` (m²) e `Weight` (kg), não comprimento, largura ou altura. A capacidade é **vetorial** (soma de recursos), não um empacotamento geométrico; empilhamento não é representado.

c) Coluna `Danger_Type`:

A coluna `Danger_Type` foi removida da modelagem por vacuidade. Esta coluna descreve tipos de pacotes (`type_1` e `type_2`) os quais, por considerações de segurança, não podem ser transportados juntos. Entretanto estes pacotes não são solicitados a um mesmo destino sequer uma vez, então esse encontro jamais poderia ocorrer na atual configuração.

d) Grandezas de tempo:

As grandezas — av_g, dl_g, τ_j, Δ e M — são medidas em **horas a partir de uma origem comum** (o menor `Available_Time` da instância): o dataset traz carimbos de data e hora, que precisam virar números para entrar nas restrições lineares, e a hora é a unidade coerente com v (km/h).

e) Classes:

A **classe** é o tipo de caminhão t, com (A_t, W_t, C_t). O custo por unidade de capacidade é monotônico no tamanho (0,332 por m² no caminhão de 7,6 m contra 0,248 no de 16,5 m), mas o custo **por caminhão** cresce com o porte — é esse *trade-off* que dá conteúdo ao VSBPP: para cargas pequenas, um caminhão menor é mais barato apesar de menos eficiente por unidade.

f) Janela de consolidação:

Δ **não é um dado**: é um parâmetro de política herdado do acelerador de referência. No `README` do repositório [2], ela aparece como a terceira restrição do problema:

> "A diferença de tempo disponível entre os itens disponíveis mais cedo e mais tarde para um mesmo caminhão deve ser definida pelo usuário (exemplo: 4 horas)" [Tradução nossa]

Adotamos o valor sugerido, $\Delta = 4$ h. As restrições 2 e 4 do mesmo `README` fundamentam, respectivamente, R7 (o caminhão só parte quando todos os pacotes estão disponíveis) e R9 (entrega antes do prazo). Não fosse especificado, um caminhão poderia esperar dias para consolidar carga. No mais, o dataset não precifica espera; por isso o modelo espera até o limite dos prazos (R8–R9), e essa ausência é apenas registrada.

3) Variáveis de decisão:

- $z_{gj} \in \{0, 1, \ldots, n_g\}$ (inteiro): quantos pacotes do grupo g viajam no caminhão j;
- $u_{gj} \in \{0, 1\}$: 1 se ao menos um pacote do grupo g viaja no caminhão j, 0 c.c.;
- $y_{jt} \in \{0, 1\}$: 1 se o caminhão j é do tipo t, 0 c.c.;
- $\tau_j \geq 0$ (h): instante de partida do caminhão j.

O modelo decide **o empacotamento** (quantos pacotes de cada grupo viajam juntos), o **tipo** de cada caminhão e **quando** ele parte. O binário u_{gj} não é uma segunda decisão: é o indicador de ocupação que as restrições temporais precisam, já que a janela depende de *haver* carga do grupo no caminhão, e não de quantas unidades há. Quando $n_g = 1$, $u_{gj} = z_{gj}$ e a formulação volta a ser a versão item a item. Os slots são idênticos antes da escolha do tipo, o que cria simetria de permutação — quebrada por R10.

4) Função objetivo:

$$\min \sum_{j \in J} \sum_{t \in T} C_t \; y_{jt} \tag{FO}$$

Parâmetro	Descrição	Coluna	Unidade
g	grupo de pacotes idênticos (derivado das quatro colunas seguintes)	—	—
a_g	área de um pacote do grupo g (≥ 0)	`Area`	m^2
w_g	peso de um pacote do grupo g (> 0)	`Weight`	kg
av_g	disponibilidade dos pacotes do grupo g (≥ 0)	`Available_Time`	h
dl_g	prazo dos pacotes do grupo g (≥ 0)	`Deadline`	h
—	origem da rota (constante `City_61`)	`Source`	—
—	destino da rota (define a instância)	`Destination`	—
—	identificador de pedido; não entra no modelo	`Order_ID`	—
—	identificador de material; não entra no modelo	`Material_ID`	—
—	classe de perigo; excluída por vacuidade	`Danger_Type`	—

Tabela III: Colunas de `order_large.csv` e `order_small.csv`: 10 colunas, 5 delas viram parâmetros de grupo.

Parâmetro	Descrição	Coluna	Unidade
—	origem (confirma `City_61`)	`Source`	—
—	destino (confirma a instância)	`Destination`	—
d	distância da rota (> 0)	`Distance(M)`	km

Tabela IV: Colunas de `distance.csv`: apenas `Distance(M)` vira parâmetro.

Parâmetro	Descrição	Coluna	Unidade
t	índice do tipo de caminhão	`truck_type`	—
A_t	capacidade de área do caminhão do tipo t (> 0)	`area_capacity_m2`	m^2
W_t	capacidade de peso do caminhão do tipo t (> 0)	`weight_capacity_kg`	kg
c_t	custo por quilômetro do tipo t (> 0)	`cost_per_km`	US$/km
v	velocidade do caminhão	`speed_kmh`	km/h
—	comprimento (usado apenas para derivar/validar A_t)	`inner_length_m`	—
—	largura (usado apenas para derivar/validar A_t)	`inner_width_m`	—

Tabela V: Colunas de `trucks.csv`: cinco parâmetros definem a frota.

Parâmetro	Descrição	Unidade
C_t	custo fixo de um caminhão do tipo t ($= c_t\, d$)	US$
t_{viagem}	tempo de viagem na rota ($= d/v$)	h
Δ	janela de consolidação ($= 4$ h)	h
M	constante de desativação ($\geq \max_g dl_g - \min_g av_g$)	h

Tabela VI: Parâmetros derivados e de política, que não vêm de coluna do dataset.

Cada caminhão aberto custa o fixo da sua classe, $C_t = c_t\ d$; caminhões fechados não custam nada. Como C_t não depende da carga, o objetivo é minimizar o **custo de frota**, e a consolidação reduz o número de veículos. O custo **não** cresce com a ocupação: para o mesmo tipo, encher um caminhão é sempre melhor que abrir outro — é o custo fixo clássico do VSBPP [8], [9].

5) Restrições:

a) Atribuição completa:

$$\sum_{j \in J} z_{gj} = n_g \quad \forall g \in G \tag{R1}$$

b) Capacidade de área:

$$\sum_{g \in G} a_g\ z_{gj} \ \leq \ \sum_{t \in T} A_t\ y_{jt} \quad \forall j \in J \tag{R2}$$

c) Capacidade de peso:

$$\sum_{g \in G} w_g\ z_{gj} \ \leq \ \sum_{t \in T} W_t\ y_{jt} \quad \forall j \in J \tag{R3}$$

As duas capacidades (R2 e R3) valem **simultaneamente**: o caminhão só é viável se respeita área e peso. Na prática a área morde primeiro — a densidade mediana dos pacotes (589 kg/m²) fica abaixo da razão do caminhão de 16,5 m (671 kg/m²), mas a cauda da distribuição passa por cima dela.

d) Um tipo por caminhão:

$$\sum_{t \in T} y_{jt} \leq 1 \quad \forall j \in J \tag{R4}$$

e) Vínculo grupo-caminhão:

$$z_{gj} \ \leq \ n_g \sum_{t \in T} y_{jt} \quad \forall g \in G,\ j \in J \tag{R5}$$

Caminhões fechados ficam vazios; por R4, o fator $\sum_t y_{jt}$ é binário.

f) Ocupação do grupo:

$$u_{gj} \ \leq \ z_{gj} \ \leq \ n_g\, u_{gj} \quad \forall g \in G,\ j \in J \tag{R6}$$

O par de desigualdades liga contagem e presença: $u_{gj} = 0$ obriga $z_{gj} = 0$ e, reciprocamente, qualquer carga positiva força $u_{gj} = 1$. É o que permite escrever as janelas em função da *presença* do grupo, sem repetir a restrição para cada cópia.

g) Partida após a disponibilidade:

$$\tau_j \ \geq \ av_g - M\ (1 - u_{gj}) \quad \forall g \in G,\ j \in J \tag{R7}$$

h) Janela de consolidação:

$$\tau_j \ \leq \ av_g + \Delta + M\ (1 - u_{gj}) \quad \forall g \in G,\ j \in J \tag{R8}$$

i) Chegada dentro do prazo:

$$\tau_j \;\le\; dl_g - t_{\text{viagem}} + M\,(1-u_{gj}) \quad \forall g \in G,\; j \in J \qquad \text{(R9)}$$

Juntas, R7–R9 caracterizam a viabilidade temporal do conjunto $S_j = \{g : u_{gj} = 1\}$ de grupos do caminhão j:

$$\max_{g\in S_j} av_g \;\le\; \tau_j \;\le\; \min\Big(\min_{g\in S_j} av_g + \Delta,\; \min_{g\in S_j} dl_g - t_{\text{viagem}}\Big),$$

ou seja, S_j é temporalmente viável se, e somente se,

$$\max_{g\in S_j} av_g - \min_{g\in S_j} av_g \le \Delta \quad \text{e} \quad \max_{g\in S_j} av_g + t_{\text{viagem}} \le \min_{g\in S_j} dl_g.$$

A escolha canônica é $\tau_j = \max_g av_g$: o caminhão parte assim que o último pacote fica pronto, o que maximiza a folga de prazo.

j) Quebra de simetria:

$$\sum_{t\in T} y_{jt} \;\ge\; \sum_{t\in T} y_{j+1,t} \quad \forall j \in \{1,\dots,m-1\} \qquad \text{(R10)}$$

R10 exige que os caminhões abertos ocupem um prefixo de J; é a ordenação já empregada em formulações de empacotamento com caixas idênticas [21] e **não exclui soluções**: dada qualquer solução viável com o caminhões abertos, renumere-os $1,\dots,o$ — trocar um caminhão aberto por um slot vazio preserva viabilidade e custo —, e o ótimo é preservado. A restrição colapsa as $m!$ ordenações de slot em uma única configuração de abertura, os $m-o$ slots vazios sendo exatamente os últimos.

O que R10 **não** faz é fixar qual caminhão aberto é qual: com o caminhões abertos, suas $o!$ permutações continuam produzindo soluções distintas de mesmo custo, que o *branch-and-bound* não consegue distinguir [17], [19]. A simetria entre **pacotes idênticos**, por outro lado, já não existe: como não há um binário por cópia, mas apenas a contagem z_{gj}, não há identidades para permutar.

6) Sumário do modelo: De forma compacta, o modelo de otimização inteira mista é:

$$\min \sum_{j\in J}\sum_{t\in T} C_t\; y_{jt} \quad \text{s.a.} \quad \text{R1–R10},$$

com $z_{gj} \in \{0,\dots,n_g\}$ inteiro, $u_{gj}, y_{jt} \in \{0,1\}$ e $\tau_j \ge 0$, e com (G,T,J), (n_g,a_g,w_g), (A_t,W_t,C_t), (av_g,dl_g), t_{viagem} e Δ conforme declarado nas subseções anteriores.

7) Complexidade computacional:

a) Tamanho do modelo:

Com $m \le n$ slots, G grupos e T tipos há $2Gm$ variáveis por grupo (uma contagem inteira e um binário de ocupação), mT binárias de tipo e m contínuas; R1–R10 somam $O(Gm)$ linhas e $O(GmT)$ coeficientes não nulos. Como $G \le n$, a formulação nunca é maior que a versão item a item, e é estritamente menor assim que houver repetição.

b) Esforço de resolução:

A relaxação linear é polinomial no tamanho do modelo; o *branch-and-bound* não é. Cada folha da árvore é uma distribuição de contagens entre os slots, e o número de distribuições é

$$\prod_{g\in G} \binom{n_g + m - 1}{m-1},$$

que coincide com m^n quando todos os $n_g = 1$ e cai rapidamente quando há repetição. O problema em si continua NP-difícil — a partição ótima em pacotes admite solução exata por programação dinâmica sobre subconjuntos em $O(n\,3^n) = 2^{O(n)}$ —, de modo que o ganho do MILP não é assintótico, e sim prático: limitante da relaxação, cortes e presolve podam a árvore [7].

c) Nestas instâncias:

As 111 instâncias vão de 1 a 198 pacotes (G de 1 a 11) e o teto de slots de pré-processamento vai de 1 a 26. Na maior delas (`City_58` em 6 de abril, 198 pacotes em 11 grupos, $m = 16$), a formulação agregada tem 432 variáveis e 1.130 linhas; a versão item a item, **com o mesmo** m, teria 3.248 variáveis e 12.933 linhas — sete e onze vezes mais. Sem o teto de slots ($m = n$), a versão item a item chegaria a 40.194 variáveis e 157.805 linhas. No conjunto, os 4.635 pacotes viram 239 grupos (instância × classe).

III. Disponibilidade de dados e código

Os dados brutos utilizados estão versionados em `data/raw/`, sem modificação em relação às fontes originais, e são distribuídos sob licença MIT pelo espelho no Kaggle [3] e pelo repositório de origem [2]. O código de modelagem e de solução, os scripts de geração de instâncias e de benchmark (`scripts/`), o registro de experimentos e o código-fonte deste relatório (Quarto) estão disponíveis em https://github.com/o-abreu/bin-packing-problem. O ambiente computacional é declarativo (Nix e devenv), o que fixa as versões de Python, dos solvers, do Quarto e da distribuição LaTeX usada na compilação: as tabelas e os números relatados são reprodutíveis a partir do repositório.

IV. Declaração de uso de inteligência artificial

Este trabalho empregou ferramentas de inteligência artificial (IA) generativa como **apoio** a atividades de pesquisa, sempre sob supervisão humana e com responsabilidade integral do autor pelo conteúdo final. Esta seção declara quais ferramentas foram usadas, como foram configuradas e quais controles existem sobre o seu uso.

A. Escopo do uso

A IA foi utilizada em: (i) busca, triagem e conferência de literatura (identificação de trabalhos, verificação de DOI e de dados bibliográficos); (ii) apoio à implementação e à depuração do código de modelagem e de solução (PuLP/HiGHS) e dos scripts de tratamento de instâncias; (iii) revisão de redação, consistência terminológica e conversão de unidades;

e (iv) checagem de consistência interna entre texto, tabelas e resultados reportados.

Não foram produzidos por IA os dados, a instância, os resultados numéricos nem as conclusões: toda solução reportada provém de execuções dos solvers sobre os arquivos em `data/raw/`, reproduzíveis a partir do código versionado, e cada número citado no texto foi reconferido contra a saída correspondente. A formulação do modelo, a escolha das instâncias e a interpretação dos resultados são de responsabilidade humana, assim como a decisão final sobre o que consta no relatório.

B. Ambiente e configuração

Todo o uso ocorreu por meio do **OpenCode**, um *agent harness* de código aberto executado localmente, cuja configuração é versionada de forma declarativa (Nix + Home Manager) no repositório público `o-abreu/nix-config`, no estado do commit `90b48b2` (28 out. 2026).[1] A configuração do agente é, portanto, auditável: os caminhos abaixo são relativos a `home/features/programs/cli/opencode/`.

- `default.nix` — habilitação do OpenCode e preferências da interface;
- `agents.nix` — instruções globais do agente (`AGENTS.md`), incluindo a regra de **nunca** instalar pacotes de forma imperativa, restringindo o ambiente ao que está declarado no flake;
- `agents/primary/research.nix` — definição do agente `research`, o único autorizado a acionar o ferramental de pesquisa bibliográfica;
- `tools/asta/default.nix` e `tools/asta/_skills.nix` — *skills* e CLI do **Asta** (AllenAI), *vendorizados* do projeto `allenai/asta-plugins`;
- `tools/zotero.nix` — acesso ao **Zotero** local via `zotero-mcp`/`zotero-cli` sobre a API HTTP local do Zotero;
- `mcp/` — definição dos servidores MCP habilitados;
- `providers/` — provedores de modelos de linguagem.

As tarefas de pesquisa usam o agente `research`, que tem permissão explícita apenas para as ferramentas de literatura; as mesmas ferramentas (Asta, Zotero e Kaggle) são negadas globalmente para os demais agentes. Essa separação é deliberada: ela concentra em um único perfil, com escopo restrito, o acesso a bases bibliográficas e a execução de comandos de pesquisa.

C. Recuperação de literatura

A busca bibliográfica usou as *skills* Asta `find-literature`, `semantic-scholar`, `pdf-download` e `literature-report`, apoiadas na API do Semantic Scholar e em PDFs de acesso aberto, além da *skill* `check-claims` para revisar a aderência entre afirmações e evidências. A bibliografia foi mantida na coleção local do Zotero *Programação Matemática — Bin Packing (VSBPP)*, com subcoleção *Datasets* para os dados e *softwares*. Toda entrada do `refs.bib` foi verificada quanto a autoria, veículo, ano e DOI; referências sugeridas por IA e não confirmadas em fonte primária foram descartadas.

D. Servidores MCP e modelos

Os servidores MCP habilitados no ambiente são `context7` (documentação de bibliotecas), `kaggle` (acesso ao conjunto de dados), `github`, `nixos`, `devenv` e `morph-mcp`. Os modelos de linguagem utilizados foram o **DeepSeek V4.1 Flash** e o **GLM 5.3 Flash**, acessados por API por meio dos provedores declarados em `providers/` (`deepseek` e `opencode-go`); a escolha entre os dois modelos é feita por sessão.

E. Controles e verificação

Três controles foram adotados: (i) ambiente reprodutível — dependências declaradas (`devenv.nix`, `requirements.txt`) e nenhuma instalação imperativa, de modo que todo código gerado com apoio de IA é executado no mesmo ambiente descrito no repositório; (ii) verificação de fontes — nenhuma referência bibliográfica entra no texto sem conferência em fonte primária; (iii) reconferência de resultados — os números do relatório são lidos das saídas do solver, não transcritos de respostas do modelo.

O autor revisou integralmente o texto, o código e os resultados aqui apresentados. Eventuais erros remanescentes são de sua responsabilidade.

[1] Repositório público: https://github.com/o-abreu/nix-config. Configuração do agente em `home/features/programs/cli/opencode/`; regras específicas deste projeto em `AGENTS.md`, na raiz do repositório do trabalho.

V. Referências

[1] I. Dzalbs e T. Kalganova, "Accelerating Supply Chains with Ant Colony Optimization Across a Range of Hardware Solutions", *Computers & Industrial Engineering*, vol. 147, p. 106610, 2020.

[2] Microsoft, "Large-Scale Route Optimization Accelerator". GitHub, https://github.com/microsoft/dstoolkit-route-optimization, 2022.

[3] mexwell, "Large-Scale Route Optimization Dataset". Kaggle, https://www.kaggle.com/datasets/mexwell/large-scale-route-optimization, 2026.

[4] R. M. Karp, "Reducibility among Combinatorial Problems", Princeton University, Complexity of Computer Computations, Plenum Press, 1972.

[5] M. R. Garey e D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York: W. H. Freeman, 1979.

[6] G. Dósa, "The Tight Bound of First Fit Decreasing Bin-Packing Algorithm Is $\mathrm{FFD}(I) \leq 11/9\,\mathrm{OPT}(I) + 6/9$", em *Combinatorics, Algorithms, Probabilistic and Experimental Methodologies (ESCAPE 2007)*, 2007, vol. 4614, p. 1–11.

[7] S. Martello e P. Toth, "Lower Bounds and Reduction Procedures for the Bin-Packing Problem", *Discrete Applied Mathematics*, vol. 28, p. 59–70, 1990.

[8] D. K. Friesen e M. A. Langston, "Variable Sized Bin Packing", *SIAM Journal on Computing*, vol. 15, nº 1, p. 222–230, 1986.

[9] J. Kang e S. Park, "Algorithms for the Variable Sized Bin Packing Problem", *European Journal of Operational Research*, vol. 147, nº 2, p. 365–372, 2003.

[10] F. D. Murgolo, "An Efficient Approximation Scheme for Variable-Sized Bin Packing", *SIAM Journal on Computing*, vol. 16, nº 1, p. 149–161, 1987.

[11] L. Epstein e A. Levin, "An APTAS for Generalized Cost Variable-Sized Bin Packing", *SIAM Journal on Computing*, vol. 38, nº 1, p. 411–428, 2008.

[12] C. Chekuri e S. Khanna, "On Multidimensional Packing Problems", *SIAM Journal on Computing*, vol. 33, nº 4, p. 837–851, 2004.

[13] Q. Hu, L. Wei, e A. Lim, "The two-dimensional vector packing problem with general costs", *Omega*, vol. 74, p. 59–69, 2018.

[14] G. J. Woeginger, "There is no asymptotic PTAS for two-dimensional vector packing", *Information Processing Letters*, vol. 64, nº 6, p. 293–297, 1997.

[15] W. Fernandez de la Vega e G. S. Lueker, "Bin packing can be solved within $1 + \varepsilon$ in linear time", *Combinatorica*, vol. 1, nº 4, p. 349–355, 1981.

[16] N. Karmarkar e R. M. Karp, "An efficient approximation scheme for the one-dimensional bin-packing problem", em *23rd Annual Symposium on Foundations of Computer Science (SFCS 1982)*, 1982, p. 312–320.

[17] F. Margot, "Symmetry in Integer Linear Programming", em *50 Years of Integer Programming 1958–2008*, M. Jünger, T. M. Liebling, D. Naddef, G. L. Nemhauser, W. R. Pulleyblank, G. Reinelt, G. Rinaldi, e L. A. Wolsey, Orgs. Berlin: Springer, 2010, p. 647–686.

[18] H. D. Sherali e J. C. Smith, "Improving Discrete Model Representations via Symmetry Considerations", *Management Science*, vol. 47, nº 10, p. 1396–1407, 2001.

[19] J. Ostrowski, J. Linderoth, F. Rossi, e S. Smriglio, "Orbital Branching", *Mathematical Programming*, vol. 126, nº 1, p. 147–178, 2011.

[20] V. Kaibel e M. E. Pfetsch, "Packing and Partitioning Orbitopes", *Mathematical Programming*, vol. 114, nº 1, p. 1–36, 2008.

[21] K. H. Salem e Y. Kieffer, "An Experimental Study on Symmetry Breaking Constraints Impact for the One Dimensional Bin-Packing Problem", em *Proceedings of the 2020 Federated Conference on Computer Science and Information Systems (FedCSIS)*, 2020, vol. 21, p. 317–326.

[22] E. G. Coffman Jr., M. R. Garey, e D. S. Johnson, "Approximation Algorithms for Bin Packing: A Survey", em *Approximation Algorithms for NP-Hard Problems*, D. S. Hochbaum, Org. Boston: PWS Publishing, 1997.